Troubleshooting

Sometimes things just don’t work the way you expect, either when testing your code in Cypress or when just running it in the browser. Browsers have a lot of different tools to allow you to explore what is happening. We’ll take a look at those tools first, then look at debugging Cypress tests in the Cypress test runner. We will end by taking a quick peek at a browser extension to troubleshoot React components.

Using the Console Log

If you are me, logging has always been a big part of your debugging flow. Visual debuggers are nice, but being able to send “yep, the code got here” and “the username is Fred” messages to the console are pretty powerful ways to understand what’s going on in your code. The console in your browser has a lot of helpful tools to make your logs more valuable. Here’s a survey.

The big one, of course, is console.log, which takes an arbitrary number of arguments and prints them to the console. If the argument is an object or an array, usually the browser provides triangle show-hide buttons for the structure of the object. Multiple arguments are printed to the console side by side. Log also allows you to use string substitution as a replacement for a template string, so you can do either ‘ console.log(User: ${user}\ ) ‘ or console.log("User: %o", user), where %o is a placeholder for an object—you also have %s for strings and %i or %f for numbers. The template strings are usually shorter, but some browsers display objects better using the string substitution method.

The log method has a bunch of friends: debug, error, info, and warn. These methods all behave the exact same way; however, the browser might choose to use a slightly different style when displaying them. For example, Chrome color-codes error and warn, and also automatically adds a stack trace to the console when those are used. Browsers typically also allow you to filter based on which method is used to send the message to the console.